

```
timescale 1ns / 1ps
`default_nettype none
```

```
module hazard_unit_tb;
```

```
    // Testbench signals
```

```
    reg [4:0] is_Rs1, is_Rs2, a1, a2;
```

```
    reg ex_regsrc, m_regsrc;
```

```
    reg [4:0] id_rd, ex_rd, mem_rd;
```

```
    reg load;
```

```
    reg amo_en_ex, amo_en_mem, amo_en_id;
```

```
    wire load_use_hazard;
```

```
    wire [1:0] mux1_sel, mux2_sel;
```

```
    // Instantiate the hazard unit
```

```
    hazard_unit dut (
```

```
        .is_Rs1(is_Rs1),
```

```
        .is_Rs2(is_Rs2),
```

```
        .a1(a1),
```

```
        .a2(a2),
```

```
        .ex_regsrc(ex_regsrc),
```

```
        .m_regsrc(m_regsrc),
```

```
        .id_rd(id_rd),
```

```
        .ex_rd(ex_rd),
```

```
        .mem_rd(mem_rd),
```

```
        .load(load),
```

```
        .amo_en_ex(amo_en_ex),
```

```
        .amo_en_mem(amo_en_mem),
```

```
        .amo_en_id(amo_en_id),
```

```
        .load_use_hazard(load_use_hazard),
```

```
        .mux1_sel(mux1_sel),
```

```
        .mux2_sel(mux2_sel)
```

```
    );
```

```
    // Stimulus
```

```
    initial begin
```

```
        $dumpfile("hazard_unit_tb.vcd");
```

```
        $dumpvars(0, hazard_unit_tb);
```

```
        // Test Case 1: No hazard
```

```
        is_Rs1 = 5'd1; is_Rs2 = 5'd2;
```

```
        a1 = 5'd1; a2 = 5'd2;
```

```
        ex_regsrc = 0; m_regsrc = 0;
```

```
        id_rd = 5'd0; ex_rd = 5'd0; mem_rd = 5'd0;
```

```
        load = 0; amo_en_ex = 0; amo_en_mem = 0; amo_en_id = 0;
```

```
        #10;
```

```
// Test Case 2: Load-Use Hazard Detected
id_rd = 5'd1; load = 1;
#10;

// Test Case 3: Forwarding from EX stage
load = 0; ex_regsrsrc = 1; ex_rd = 5'd1;
#10;

// Test Case 4: Forwarding from MEM stage
ex_regsrsrc = 0; m_regsrsrc = 1; mem_rd = 5'd1;
#10;

// Test Case 5: AMO instruction in EX stage (should forward)
m_regsrsrc = 0; amo_en_ex = 1; ex_rd = 5'd1;
#10;

// Test Case 6: AMO instruction in MEM stage (should forward)
amo_en_ex = 0; amo_en_mem = 1; mem_rd = 5'd2;
#10;

// Test Case 7: AMO instruction in ID stage (should cause stall)
amo_en_mem = 0; amo_en_id = 1; id_rd = 5'd2;
#10;

$display("Testbench completed.");
$finish;
```

```
end
```

```
endmodule
```